

L11 Multi-Agent 总结

L11 多智能体协作 · 课程总结

封版日期：2026-08-04 (Windows) · 对标书 Ch10 · 书主线课。交付物：multi_agent_review.py (多 Agent 代码审查系统) + test_multi_agent_review.py (4 测试)。定位：L4 编排 + reflection(actor-critic) 的自然延伸；边投边学第一课。

一、这节课学了什么（一句话）

多 Agent 不是“多几个 loop 更聪明”，而是**两个正交维度的架构选型**（上下文共享/隔离 × 协作拓扑）；是否值得，只看一条硬判据：**协作有没有引入单 Agent 拿不到的新信息**——没有就别上。

二、核心框架（书 Ch10）

维度一：上下文共享 vs 隔离 = 线程 vs 进程

	共享（线程）	不共享（进程）
后者看到	前者完整轨迹	只看显式传的
隔离/并行	无隔离、难并行	强隔离、可并行
成本	单轨迹累积	总 token 常高数倍~一个量级

- 通信（不共享时）= IPC 两范式：共享文件系统(=共享内存) / 工具参数消息总线(=消息传递)。
- 经验法则：上下文 > 窗口 50% → 隔离；信息零损耗硬约束 → 共享；多数系统“阶段切换式”。

维度二：协作拓扑（不共享时才有设计）

对等协作(Peer, 2-3 迭代改进) / 管理者(Orchestration, 中心 Manager+子 Agent) / 去中心化(handoff/group chat/A2A)。

★ 判据：何时真正优于单 Agent（面试必问）

唯一核心：协作是否引入了单 Agent 生成时无法获得的新信息？ - 引入了（执行反馈/视觉/工具验证）→ 显著提升（RLEF、WebGen-Agent）。 - 没引入（同模型自审、纯辩论）→ 通常无效甚至有害。 - **成本**：Anthropic 多 Agent 研究系统 ≈ 15× token，收益必须盖得住。

失败模式

① 共享文件系统并发冲突 ② 错误级联放大（上游错判被下游当权威）。

三、★ 打脸呼应：上节的 reflect 正好是“无效的那种”

书表10-2 第一行“同一模型自我审查=通常无效甚至有害”——这正是上节 **reflect** 的形态！这解释了为什么 reflect 端到端 6 场景全 **accept**：它没引入新信息，只让同模型重看自己写的话。> 我“该不该每轮反思”的工程直觉，被 Ch10 的判据从理论上印证了。要让自审有效，得喂它 actor 拿不到的**新信息**（真调工具验证/换更强模型/外部规则）。

四、动手：多 Agent 代码审查系统（自己提的项目）

架构 = 管理者模式 + 不共享上下文：Manager 并行分派 diff 给多个隔离 reviewer → 汇总报告。

为什么这个项目“有效”（踩对判据）：至少一个 reviewer 带**真实外部反馈**—— - **static_review**：跑 **pyflakes**（工具反馈）抓未定义名/未用 import = 确定性 ground-truth。 - **llm_review(安全/正确性)**：LLM lens，读代码找逻辑/安全问题。

活证据（互补性，面试杀招）：同一段代码—— - pyflakes 抓到 **db** 未定义、**os** 未用（LLM 漏了）； - LLM 抓到 SQL 注入、除零（工具看不见）。 - **价值来自 reviewer 的异质性，不是人头数**。堆 3 个同样的 LLM lens 抓的还是同一类。

四个组件：| 组件 | 要点 | |——| | **static_review** | subprocess 跑 pyflakes；**sys.executable**（不是裸 python）；try/finally 删临时文件；解析先剥已知路径再切（绕 Windows 盘符冒号） || **llm_review** | 隔离上下文（全新 [system,user]）+ 依赖注入 + json_object + fail-open 返回 [] || **synthesize** | 纯逻辑：去重 + **工具证据排 LLM 意见前**（\$4 防级联） || **review** | ThreadPoolExecutor **并行**（I/O-bound 用线程）+ flatten + 交 synthesize |

五、这次课踩的坑（都是真实工程故事）

1. **sys.executable** vs 裸 **python**：subprocess 调 pyflakes 时裸 **python** 跑到没装 pyflakes 的别的解释器（“哪个 python”老坑）→ 用 **sys.executable**。
2. **Windows 盘符冒号打乱解析**：**c:\...** 的冒号让 **split(":")** 错位、行号全丢。**修法/原则**：**解析工具输出时，已知的东西（路径）先剥掉，只解析未知部分**。
3. **prompt↔parser 契约不一致**：改了 prompt 格式却没和 **data.get("findings")** 对齐 → findings 被丢光。**同一契约写在两处必须同步**。还发现 **source** 不该问模型（它填成了函数名）→ 由代码盖章。
4. **NamedTemporaryFile** 少 **mode="w"**：默认二进制模式不接受 encoding。
5. **测试断言对着“我以为”而非“真实输出”**：断言中文语义，但 pyflakes 说英文 → 红。**assert 要对着工具实际吐什么**。
6. **并行正确性**：先全 submit 再收集 result（否则退化串行）；**list.append** 在 GIL 下原子，共享结构不用自己加锁。

六、门禁三条（全过）

- 环境可复现 ☒ (uv + pyflakes pin)
- 4 测试正反俱全 ☒: synthesize 去重排序 / static 确定性抓 bug / **隔离性(招牌)** / fail-open。
- 无残留 ☒ (你来写·TODO 清光、print→logger、py_compile 过)。

七、Quiz (5/5)

线程/进程类比 · **reviewer agent 大概率无效除非引入新信息** · 辩论 vs 执行反馈的区别=有没有外部 ground-truth · 何时选隔离 · 客服 capstone 不该拆四 agent (判据同 reflection 门控)。

八、接到项目 & 下一步

- 客服 capstone 该上多 Agent 吗? **判据自检 = 不该** (拆专家不引入新信息, 纯加成本)。和 reflection 门控同源判断。
- 本系统可扩展 (真正的“执行反馈”版): 让 reviewer 真跑测试套件 (RLEF 形态)。
- 语义去重 (现字符串级, 同义合不掉)、动态步骤预算 (书 §3) = 未来增强。

一句话简历版: 做了个多 Agent 代码审查系统 (管理者模式+隔离上下文+并行), 刻意让 reviewer 带**真实工具反馈** (pyflakes)而非堆同模型 LLM——因为我知道 N 个同模型 reviewer 读同一段 diff 不引入新信息、等于白烧钱。价值来自异质信息源, 不是人头数。